

Data Frame Selection and Indexing

We've seen how to call built-in data frames and how to create them using `data.frame()` along with vectors. Let's revisit our weather data frame and learn how to select elements from within the dataframe using bracket notation:

In [1]:

```
# Some made up weather data
days <- c('mon','tue','wed','thu','fri')
temp <- c(22.2,21,23,24.3,25)
rain <- c(TRUE, TRUE, FALSE, FALSE, TRUE)

# Pass in the vectors:
df <- data.frame(days,temp,rain)
```

In [2]:

df

Out[2]:

	days	temp	rain
1	mon	22.2	TRUE
2	tue	21	TRUE
3	wed	23	FALSE
4	thu	24.3	FALSE
5	fri	25	TRUE

We can use the same bracket notation we used for matrices:

df[rows,columns]

In [4]:

```
# Everything from first row
df[1,]
```

Out[4]:

	days	temp	rain
1	mon	22.2	TRUE

In [5]:

```
#Everything from first column
df[,1]
```

Out[5]:

mon tue wed thu fri

In [6]:

```
# Grab Friday data
df[5,]
```

Out[6]:

	days	temp	rain
5	fri	25	TRUE

Selecting using column names

Here is where data frames become very powerful, we can use column names to select data for the columns instead of having to remember numbers. So for example:

In [8]:

```
# All rain values  
df[, 'rain']
```

Out[8]:

```
TRUE TRUE FALSE FALSE TRUE
```

In [11]:

```
# First 5 rows for days and temps  
df[1:5, c('days', 'temp')]
```

Out[11]:

	days	temp
1	mon	22.2
2	tue	21
3	wed	23
4	thu	24.3
5	fri	25

If you want all the values of a particular column you can use the dollar sign directly after the dataframe as follows:

df.name\$column.name

In [12]:

```
df$rain
```

Out[12]:

```
TRUE TRUE FALSE FALSE TRUE
```

In [15]:

```
df$days
```

Out[15]:

```
mon tue wed thu fri
```

You can also use bracket notation to return a data frame format of the same information:

In [14]:

```
df['rain']
```

Out[14]:

	rain
1	TRUE
2	TRUE
3	FALSE
4	FALSE
5	TRUE

In [18]:

```
df['days']
```

Out[18]:

	days
1	mon
2	tue
3	wed
4	thu
5	fri

Filtering with a subset condition

We can use the **subset()** function to grab a subset of values from our data frame based off some condition. So for example, imagin we wanted to grab the days where it rained (rain=True), we can use the **subset()** function as follows:

In [19]:

```
subset(df, subset=rain==TRUE)
```

Out[19]:

	days	temp	rain
1	mon	22.2	TRUE
2	tue	21	TRUE
5	fri	25	TRUE

Notice how the condition uses some sort of comparison operator, in the above case ==. Let's grab days where the temperature was greater than 23:

In [20]:

```
subset(df, subset= temp>23)
```

Out[20]:

	days	temp	rain
4	thu	24.3	FALSE
5	fri	25	TRUE

Another thing to note is that we didn't pass in the column name as a character string, subset knows that you are referring to a column in that data frame.

Odering a Data Frame

We can sort the order of our data frame by using the **order** function. You pass in the column you want to sort by into the **order()** function, then you use that vector to select from the dataframe. Let's see an example of sorting by the temperature:

In [28]:

```
sorted.temp <- order(df['temp'])
```

In [29]:

```
df[sorted.temp,]
```

Out[29]:

	days	temp	rain
2	tue	21	TRUE
1	mon	22.2	TRUE
3	wed	23	FALSE
4	thu	24.3	FALSE
5	fri	25	TRUE

Let's take a look at what sorted.temp actually is:

In [30]:

```
sorted.temp
```

Out[30]:

```
2 1 3 4 5
```

Ok, so we are just asking for those index elements in that order (by default ascending, we can pass a negative sign to do descending order):

In [31]:

```
desc.temp <- order(-df['temp'])
```

In [32]:

```
df[desc.temp,]
```

Out[32]:

	days	temp	rain
5	fri	25	TRUE
4	thu	24.3	FALSE
3	wed	23	FALSE
1	mon	22.2	TRUE
2	tue	21	TRUE

We could have also used the other column selection methods we learned:

In [34]:

```
sort.temp <- order(df$temp)
df[sort.temp,]
```

Out[34]:

	days	temp	rain
2	tue	21	TRUE
1	mon	22.2	TRUE
3	wed	23	FALSE
4	thu	24.3	FALSE
5	fri	25	TRUE

That's it for data frames! We will definitely revisit this and explore data frames A LOT more, but we should test your understanding first! Up next an exercise!